

Part 4

Machine learning on knowledge graphs

This part of the book explores how representation learning and graph neural networks can transform the static knowledge contained in graphs into dynamic, learnable features:

- Neural-network-based representations capture the complexity of graph structures and their entities.
- Structured information can be effectively encoded in vector spaces.
- Flexible feature representations support downstream tasks, from classification to link prediction.
- Interpretation of learned embeddings allows automated knowledge extraction.

Chapter 9 introduces the fundamental concepts and motivations for applying machine learning (ML) to KGs, establishing why graph-based approaches better reflect real-world dependencies and how they can be applied to computational tasks.

Chapter 10 illustrates manual and semiautomated approaches to feature engineering in graph-based ML, demonstrating how graph metrics and structural patterns can be captured and utilized.

Chapter 11 shows how graph neural networks can automatically learn optimal representations from graph structures.

Chapter 12 demonstrates these concepts in action through two real-world implementations that showcase how graph neural networks can tackle business challenges while maintaining interpretability.

Many representation learning techniques mirror approaches used in modern language models. This parallel highlights the broader theme of our book: the combination of structured knowledge representation with advanced ML techniques.

Machine learning on knowledge graphs: A primer approach

This chapter covers

- Understanding machine learning on knowledge graphs
- Exploring common machine learning tasks performed on graphs
- Understanding the role of node and relationship representations

Building knowledge graphs is a crucial step in developing intelligent systems. It enables us to acquire holistic knowledge from multiple and diverse data sources, representing it in a way that supports exploration, navigation, and more advanced analytics. So far, we have seen how to query a graph and extract relevant information, how to navigate through nodes and relationship types, and even how to extract statistical information to validate the import process and evaluate the “quality” of the knowledge stored in the KG. These are all important steps in building *intelligent advisory systems* (IASs).

In an IAS, “advising” provides insights that users cannot extract on their own. For example, how can a researcher efficiently navigate a vast KG containing diseases, proteins, genes, and compounds to identify potential opportunities for repurposing drugs? Or, how can a clinician combine patients’ symptoms and DNA sequences with literature, clinical trials, and standard protocols to develop personalized treatment plans? There are many such scenarios. And most, if not all, require using machine learning (ML) algorithms that take the knowledge in the graphs as input.

Chapters 9–12 delve into the realm of ML on KGs. We’ll extend our previous book [1] with new algorithms and theories. These, in the last few years, have been improved (in particular graph neural networks [GNNs]) and tested on graphs (such as LLMs). We’ll also discuss new libraries that are stable and performant enough to be considered in a production environment.

9.1 *Machine learning on graphs: Why?*

We begin by giving you a clear understanding of why executing ML on (knowledge) graphs can be valuable (with and without LLMs) and why this approach is sometimes the optimal—or even the only feasible—choice. The key reasons can be summarized as follows:

- *Data representation*—Data used or produced by real-world applications has diverse forms, from matrixes and tensors to sequences and time series [2]. Graphs provide a universal data representation; many kinds of data from a wide variety of systems can be transformed into graphs.
- *Problem modeling*—A huge number of problems can be addressed as a small set of computational tasks on graphs. For example, detecting anomalous nodes and suggesting medications for patients can be summarized as problems of node classification, and making recommendations and identifying interactions are essentially problems of relationship prediction.
- *Data item dependency*—Traditional ML algorithms assume that data items are independent and identically distributed. But in many real-world use cases, data items are intrinsically connected, and ignoring these relationships can lead to incomplete or wrong results. Storing data as graphs, which naturally store relationships, and applying computational tasks on those graphs can give better results [3].

ML techniques like GNNs together with LLMs create powerful IASs by combining the GNN’s ability to detect complex structural patterns with the LLM’s natural language understanding and generation capabilities. For example, in personalized medicine, GNNs process patient similarity networks and biological pathway graphs, while LLMs synthesize medical literature and clinical guidelines to explain treatment rationales in clear medical terminology. This GNN–LLM integration enables the system to identify complex patterns and explain them in context-appropriate language, making insights accessible and actionable for end users.

You want to use the best tool possible to achieve your final goal and solve your business problem. ML is inherently a problem-driven discipline; so for many of the scenarios we'll consider, ML on graphs will be the best arrow in your bow.

9.2 Machine learning on graphs: What?

Classical ML algorithms can be categorized in different ways. The most common approach differentiates them according to the type of data and the task they solve, classifying them as supervised or unsupervised:

- In a *supervised* algorithm, data is partially labeled: the output is known for certain data items, and the goal is to predict the labels for the remaining data. A typical example of this learning process is a spam filter. The learner requires labels, such as “spam” and “not spam” (the significant information), in the training dataset for each data item (emails). It learns from these labels how to classify an email.
- In an *unsupervised* algorithm, the data is fully unlabeled, and the goal is to extract insights and patterns from it, such as clusters from a set of data points or communities from a graph.

ML on graphs is no different, but in this case the supervised and unsupervised categories are not necessarily the most useful [4], even though they remain valid: for example, node classification can be considered a supervised task (even though it isn't, exactly), and community detection is unsupervised. Instead, ML tasks on graphs are generally divided into these two categories (see figure 9.1):

- *Node-focused tasks*—The entire dataset is represented as one graph, with nodes and relationships as the data points.
- *Graph-focused tasks*—The data consists of a set of graphs, and each data point is an entire graph.

In this section, we will go through the most important and well-studied ML tasks on graph data. We'll outline the primary tasks in each category, focusing on those that you're likely to find the most useful.

9.2.1 Node classification

Suppose you manage a social network with millions of users. Among these users are a substantial number of bots. Detecting these bots is important because they could violate the network's terms of service, spread fake news, or be irrelevant targets for marketing campaigns. Manually identifying them is infeasible. Ideally, you want a model that can differentiate users as either bots or legitimate users, given a small set of manually labeled examples. This is a classic example of a node classification task: for each unlabeled node u in graph V , the goal is to predict the label y_u , when we are only given the known labels on a training set of nodes V_{train} that is a small subset of V .

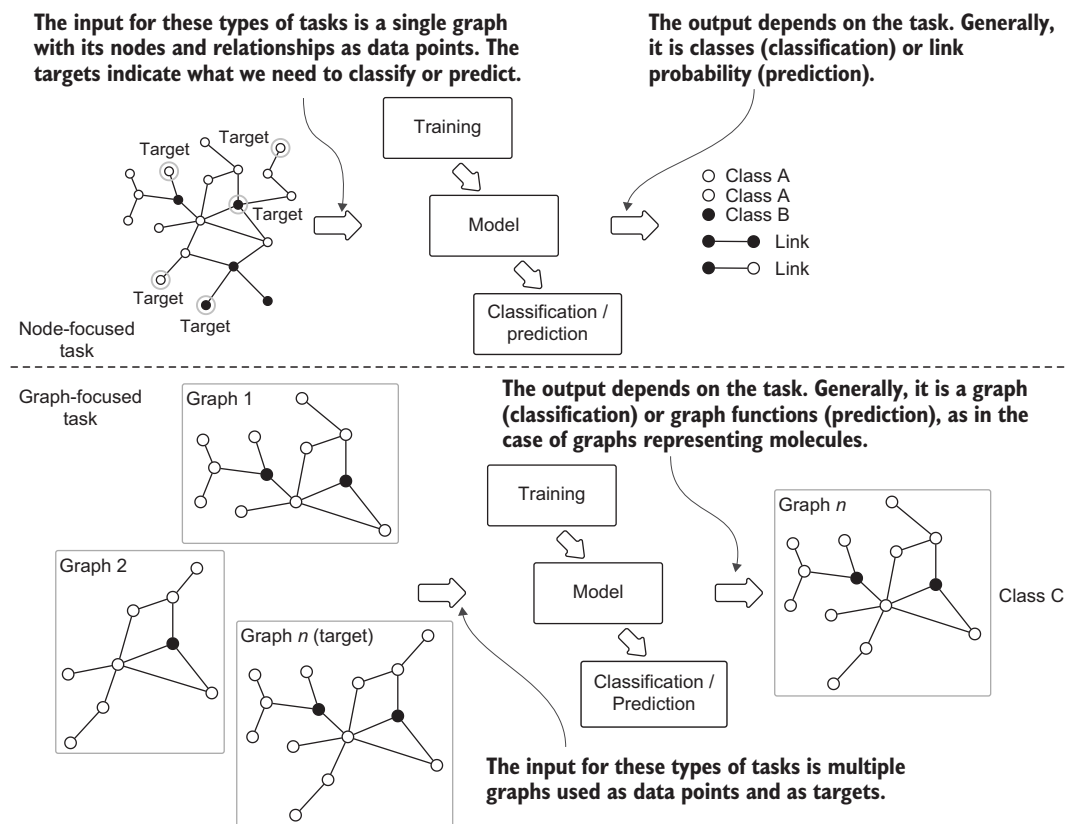


Figure 9.1 Node-focused and graph-focused tasks are represented in terms of expected input and output during training and prediction.

Other examples of node classification include classifying the function of proteins in the interactome [4] and classifying the topic of documents based on hyperlinks or citation graphs [5]. It is a powerful tool during the implementation of an IAS, helping with identification and decision-making. Figure 9.2 delineates the principal components and phases of node classification.

At first glance, node classification seems to be a straightforward variant of standard supervised classification, but there are distinct differences. Notably, the nodes in a graph are not *independent and identically distributed* (i.i.d.). Typically, in constructing supervised ML models, we assume that each data point is statistically independent of all other data points. If this is not the case, we may need to clarify the dependencies among all of our input points. Similarly, we assume that the data points are identically distributed, to ensure that our model can effectively generalize to unseen data points. But node classification shatters this i.i.d. assumption. Instead of modeling a set of i.i.d. data points, we're modeling an interconnected network of nodes.

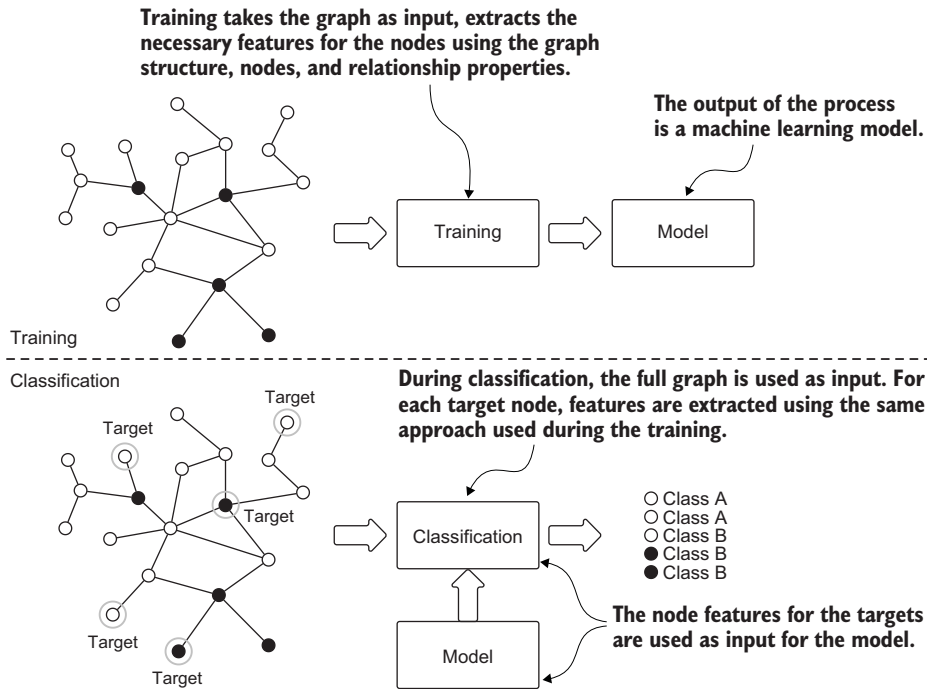


Figure 9.2 The typical flow of node classification. Like many supervised ML tasks, it has two phases: training and prediction (which, in this case, translates into classifying unclassified nodes).

Different graph types exhibit various relationship patterns that challenge the i.i.d. assumption. In social networks, *homophily* illustrates this interconnectedness: nodes influence each other through their relationships, exhibiting shared interests, attributes, and behaviors with their neighbors [6]. This violation of the i.i.d. assumption means that effective models must consider both node features and their network relationships when making predictions [7]. In protein interaction networks, structural equivalence becomes crucial—nodes with similar neighborhood structures tend to share functional properties [8]. *Heterophily* presents another pattern where nodes preferentially connect to those with different characteristics, further demonstrating how real-world data often violates i.i.d. assumptions. These principles underscore why treating nodes as independent data points fails to capture the rich relational information encoded in graphs. Successful node classification requires modeling both node attributes and their complex interdependencies.

Is node classification supervised or unsupervised?

Many researchers agree that node classification is *semisupervised* [9] because when we train node classification models, we usually have access to the full graph, including all the unlabeled (e.g., test) nodes. The only thing we are missing is the labels of

(continued)

the test nodes. However, we can still use information about the test nodes (e.g., knowledge of their neighborhood in the graph) to improve our model during training. This is different from the usual supervised setting, in which unlabeled data points are unobserved during training.

The general term for models that combine labeled and unlabeled data during training is *semisupervised learning*, so, understandably, this term is often used for node classification tasks. It is important to note, however, that standard formulations of semisupervised learning require the i.i.d. assumption, which does not hold for node classification. We still struggle with this definition. This shows why ML tasks on graphs do not easily fit classical categories!

Node classification can be expanded to assign multiple labels to a single node. For instance, Flickr (www.flickr.com), an image-hosting platform, uses graphs and multilabel node classification to address users' interests. In addition to hosting photos, Flickr serves as an online social community where users can follow each other, thus forming a network through these user connections. Moreover, Flickr users can subscribe to interest groups; these memberships signify user interests and can serve as user labels. Users can subscribe to numerous groups, so each user may be associated with multiple labels. A multilabel node classification problem on graphs can help predict potential groups that users might be interested in but haven't yet subscribed to; Tang and Liu [10] provide datasets related to such Flickr user behaviors.

9.2.2 Link prediction (a.k.a. relationship prediction)

Link prediction is a fundamental task in graph-based ML that identifies potential future connections between nodes. Consider a comprehensive database of research papers from domains such as healthcare (PubMed, <https://pubmed.ncbi.nlm.nih.gov>; MedRxiv, www.medrxiv.org), COVID-19 research (CORD-19, <https://mng.bz/JwGQ>), broad scientific research (Web of Science, <https://mng.bz/qRPz>), or computer science (DBLP, <https://dblp.org>). From these sources, we can construct a co-authorship graph in which authors represent nodes and edges represent their collaboration on at least one paper. The link prediction task then involves forecasting likely future collaborations between authors who haven't yet worked together.

This predictive approach extends naturally to other critical domains, such as law enforcement and security applications. Figure 9.3 illustrates the two primary phases of a typical link prediction task.

In many real-world applications, graphs are not complete because of missing edges. This incompleteness is usually due to one of two reasons. First, sometimes connections exist but are not observed or recorded—or, in certain cases, they are kept hidden by the key actors in the network. And second, many graphs are naturally evolving; for example, in an academic collaboration graph, an author can always build new collaboration relations with other authors by writing a new article. Inferring or

To train a machine learning model, it is also necessary to sample negative cases: links that don't exist among all the possible missing links.

Training takes the graph as input and extracts the necessary features for the relationships (existing and not) using the graph structure, nodes, and relationship properties.

The output of the process is a machine learning model.

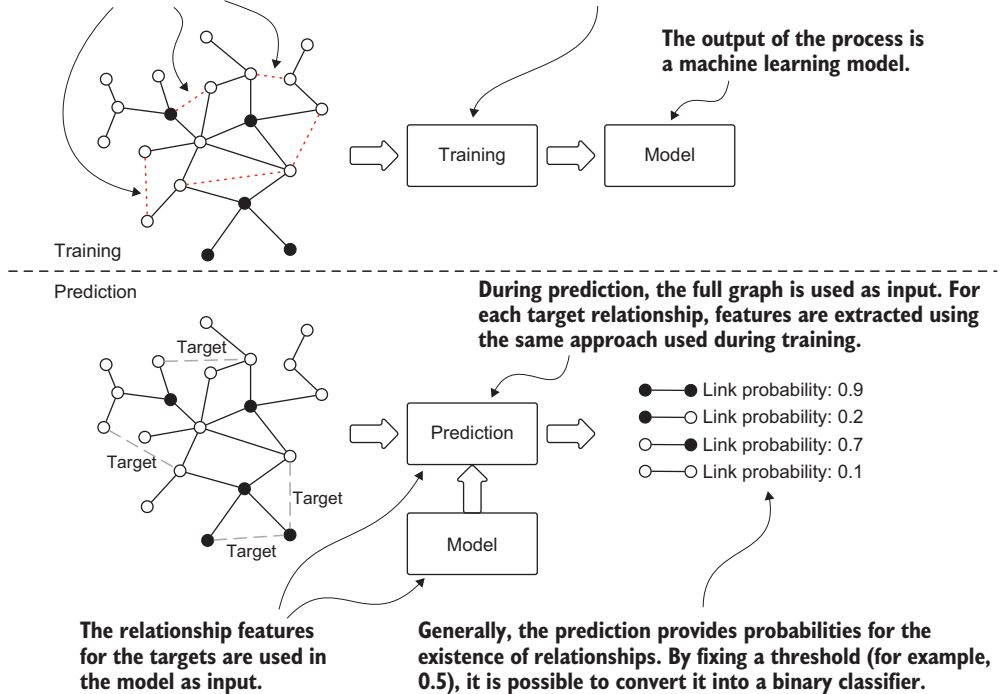


Figure 9.3 Typical link prediction flow. It has two phases. During the training phase, the graph is used as input, and the model is trained to determine whether a link exists. During the prediction phase, the model provides an output for each pair of nodes representing the target relationship to predict. It provides an existence probability for each.

predicting these missing edges can benefit many IASs, helping with friend recommendations [11, 12], product recommendations, KG completion [13], predicting drug side effects [14], inferring new facts in a relational database [15], discovering protein-protein interactions [16], analyzing criminal intelligence [17], and many more.

Names for the link prediction task

This task is known as *link prediction*, *graph completion*, or *relational inference*, among other names, depending on the application domain. *Link prediction* typically pertains to predicting the existence of a connection between two nodes, regardless of the relationship type (which is consistently the same and predetermined). Conversely, the term *relationship prediction* is generally used when we want to pinpoint not just the existence but also the type of relationships. In this book, we'll use these terms interchangeably.

Like node classification, link prediction is typically approached as a semisupervised learning task. Although specific links between node pairs may be missing, we can use both existing examples of connections and rich node-level information to predict potential relationships. This semisupervised nature allows us to use known patterns in the graph structure along with node attributes to infer likely missing or future connections between nodes.

9.2.3 Clustering and community detection

Imagine that you obtain a catalog of research papers from any of the sources mentioned in the previous section. You generate a collaborative graph connecting researchers who have coauthored papers. On inspecting this network, you're unlikely to observe a dense "hairball" where collaborations are equally probable among everyone. Instead, the graph is probably partitioned into distinct clusters of nodes grouped by such elements as research area, institution, or geographic factors. The propensity for two researchers from the same university to collaborate is higher than that of colleagues situated remotely. Similarly, two researchers in the same field are more likely to reference other researchers in that sphere than in an unrelated domain. This natural clustering behavior forms the theoretical foundation for community detection algorithms, which aim to identify these inherent groupings in the network structure (see figure 9.4).

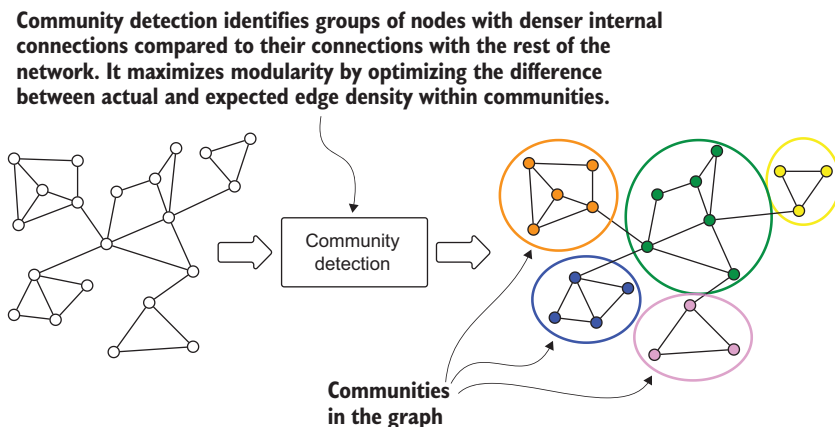


Figure 9.4 Flow of community detection. The input is a graph, and the output is a mapping between nodes and groups (circled in the image).

Community detection uncovers latent group structures in a graph using only the network's topology and relationships. This task has diverse real-world applications, from identifying functional modules in genetic interaction networks [18] to detecting fraudulent user groups in financial transaction networks [19].

One of the most powerful applications of graph clustering is in graph description and summarization. By identifying densely connected regions, clustering provides a higher-level view of the network's organization. This capability is particularly valuable when dealing with large-scale graphs that defy direct visualization or comprehensive manual analysis, effectively providing a structural summary of complex network relationships.

Names for the community detection task

The terms *community detection* and *graph clustering* are often used interchangeably in network analysis, but they differ fundamentally from traditional clustering algorithms like K-means and DBSCAN. K-means partitions n data points into k clusters by iteratively assigning each point to the nearest cluster center (centroid) and updating these centers; k must be specified in advance. DBSCAN (density-based spatial clustering of applications with noise) groups together points that are closely packed (points with many nearby neighbors) while marking points in low-density regions as outliers; it doesn't require specifying the number of clusters beforehand.

The key distinction lies in the data structure: traditional clustering algorithms work with independent data points in vector space, whereas graph clustering specifically handles interconnected data in which relationships between nodes are crucial to determining group membership. We'll use these terms interchangeably, as they refer to the same task when applied to graphs.

Graph clustering is generally unsupervised, requiring no prelabeled information to identify community structures. However, some clustering approaches, such as label propagation [20], can incorporate existing labels to guide community assignment, bridging the gap between supervised and unsupervised learning in graph analysis.

The three previous algorithms belong to the category of node-focused tasks. Next, we'll look at one that is graph-focused.

9.2.4 Graph classification

Consider the task of predicting the toxicity and solubility of chemical molecules. These properties emerge not just from individual atoms but also from how these atoms are connected to form the molecule. We can represent each molecule as a graph, where atoms serve as nodes and chemical bonds as edges [21]. This graph representation allows us to apply graph classification techniques to predict multiple molecular properties simultaneously. As illustrated in figure 9.5, graph classification can systematically analyze the atomic composition and structural patterns to categorize previously unlabeled molecules based on properties such as solubility and toxicity.

Whereas node classification predicts labels for individual nodes in a single graph, graph classification deals with datasets containing multiple independent graphs, each representing a complete sample (such as a molecule). In graph classification, each graph serves as an i.i.d. data point with its label: for instance, whether a molecule is toxic.

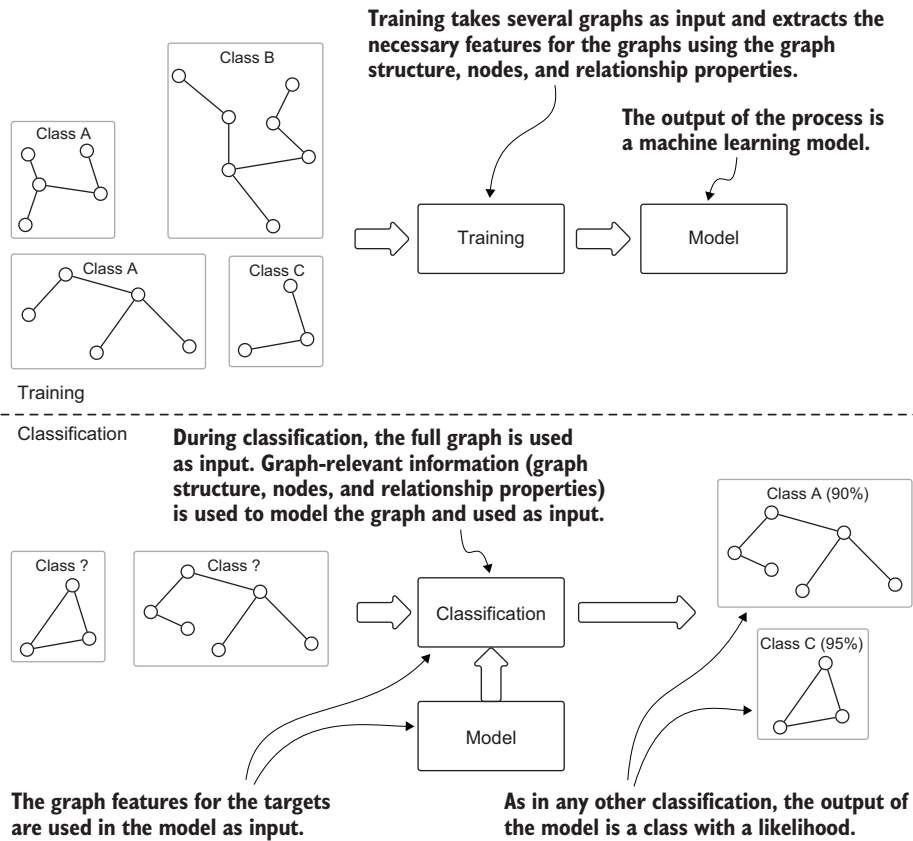


Figure 9.5 Flow of graph classification. Like many supervised tasks, it has two phases. During training, it trains different graphs with related classes to recognize the class of each. And during prediction, the model predicts the class of a graph.

The goal of graph classification is to learn a mapping from entire graphs to their associated labels using a training set of labeled graphs. Similarly, graph clustering at the graph level extends traditional unsupervised clustering to categorize entire graphs rather than nodes. The primary challenge in these graph-level tasks lies in developing features that effectively capture both the internal structure of each graph and the properties of its components.

Graph clustering has many real-world applications, such as in IASs. For example, enzymes are a type of protein. Proteins can be represented as graphs, where amino acids are nodes, and edges between two nodes can be created if they are less than a certain distance apart. After the training, given a protein, the graph classification algorithm can predict whether it is an enzyme. Another example is a malware classifier: the task in this case is to build a classification model to detect whether a computer program is malicious by analyzing a graph-based representation of its syntax and data flow [22].

9.3 Machine learning on graphs: How?

At this point, we understand why ML on graphs has emerged as a distinct branch of research and the complexities these algorithms can address. Now we'll explore our implementation options. Generally, solutions can be developed in two directions, as summarized in figure 9.6. The first approach uses algorithms specifically designed for graphs, such as collective classification [23]. These specialized algorithms directly process the graph structure, simultaneously considering both node features and neighborhood relationships to make predictions. The second approach transforms the graph problem into a traditional ML task by first converting graph structures into feature vectors. This feature engineering step allows us to use the entire ecosystem of existing ML algorithms, including modern deep learning techniques. The main challenge then shifts to defining appropriate features that capture the essential characteristics of nodes, edges, and, in the case of graph classification, the entire graph structure.

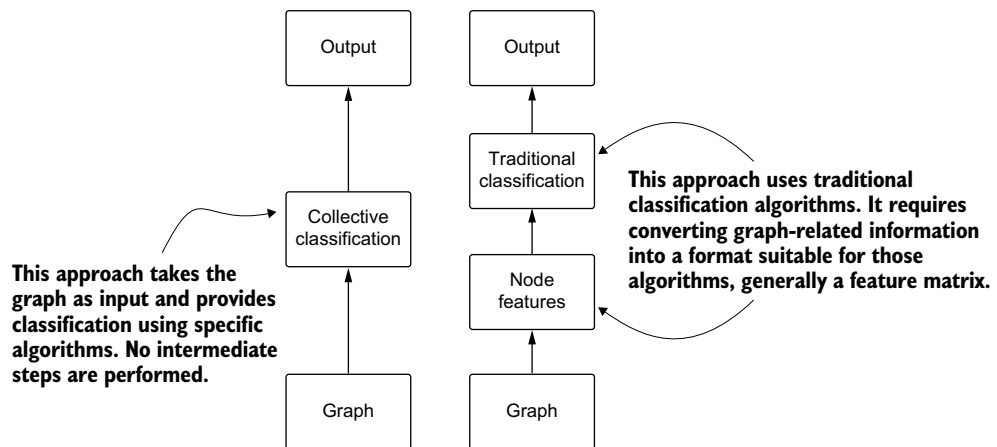


Figure 9.6 A graph-based classification approach (collective) versus traditional algorithms for classification

In this part of the book, we focus on feature engineering for graphs. We begin by exploring manual feature extraction methods, demonstrating their thoroughness but also their tedious nature. We then progress to semiautomated approaches before diving into GNNs as a powerful solution for automated feature learning. GNNs excel at capturing both structural patterns and node properties, making them particularly valuable for incomplete KGs. We'll examine how GNNs create meaningful vector embeddings that encode graph knowledge in ways that directly benefit downstream ML tasks such as classification (for node classification, for example).

9.3.1 Node classification and link prediction

Figure 9.7 shows the high-level training process for node classification and link prediction. As a result of this training, a prediction model is created: it is the output of this phase and the input of the next phase. Figure 9.8 shows the steps of the prediction process, which takes the existing nodes and relationships and predicts classes and missing links.

NOTE The featurization process used during training must be the same as that used for making predictions. Otherwise, the prediction phase will not function correctly.

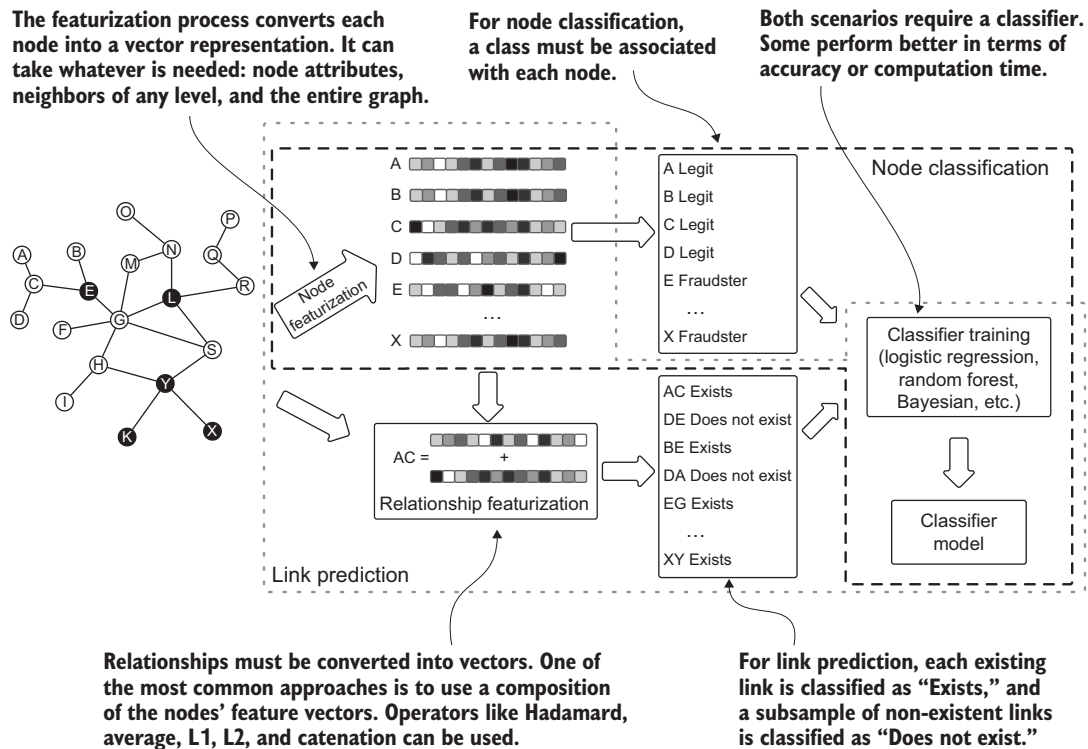


Figure 9.7 Training flow for node classification and link prediction. A critical step is the featurization of nodes and relationships. When this process is finished, the vectors can be passed to a classic algorithm. Both node classification and link prediction can be seen as classification tasks.

Let's put these principles into practice with a simple example. Suppose you would like to classify nodes in a network. For learning purposes, we will use a small graph: the famous Zachary Karate Club [24]. The graph documents the relationships among 34 members of a karate club, tracking interactions between pairs of members outside the

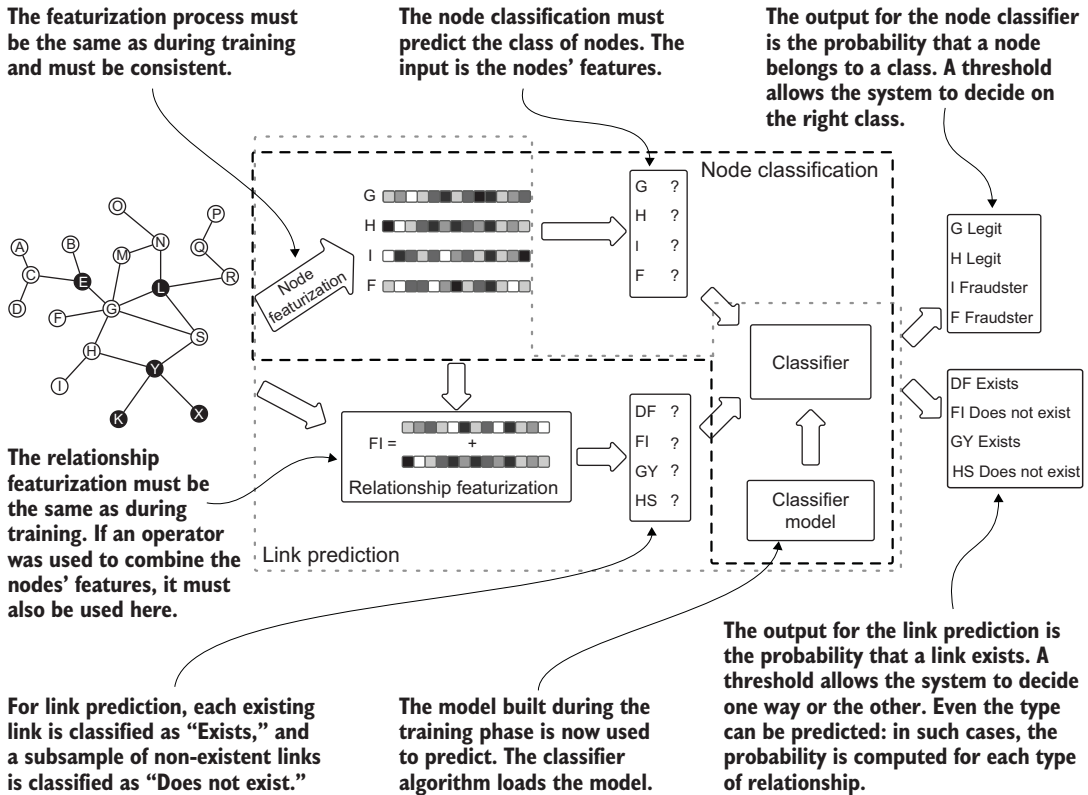


Figure 9.8 Prediction flow for node classification and link prediction. The featurization process should be the same as during training. The classifier model built previously is used in this phase to make predictions.

club. A dispute between the administrator, “John A,” and the instructor, “Mr. Hi” (pseudonyms), caused the club to divide into two factions.

After the club’s split, each member (represented as a node) either became affiliated with the instructor’s new club (Mr. Hi) or remained with the administrator’s original club (John A). This real-world outcome provides us with ground-truth labels: each node is labeled according to which club the member ultimately joined. Our node classification task is to predict these club affiliations using only the friendship network structure that existed before the split, demonstrating how network patterns can predict social behavior. We’ll explain, using simple code, the entire flow described in figures 9.7 and 9.8.

Let’s inspect the network first and then analyze it. For this example, we won’t store anything in Neo4j. The network is very small, and it is available out of the box in many network analysis tools, as well as in Networkx (<https://networkx.org>), which is the tool we will use. The following listing shows how to import the necessary packages and load the karate club graph.

Listing 9.1 Creating and drawing a karate club network

```

import networkx as nx
import matplotlib.pyplot as plt

G = nx.karate_club_graph()
draw_and_save_graph_picture(G)

def draw_and_save_graph_picture(G):
    set_club_colors(G)
    layout_position = nx.spring_layout(G, k=8 / math.sqrt(G.order()))
    colors = [n[1]['color'] for n in G.nodes(data=True)]
    nx.draw_networkx(G, pos=layout_position, node_color=colors)
    plt.axis('off')
    plt.savefig("Karate_Graph.svg", format="SVG", dpi=1000)
    plt.savefig("Karate_Graph.png", format="PNG", dpi=1000)
    plt.show()

def set_club_colors(G):
    for node in G.nodes(data=True):
        color = '#00fff9'
        if node[1]['club'] == 'Mr. Hi':
            color = '#e6e6fa'
        node[1]['color'] = color

```

← Loads the karate club graph

← Displays the graph onscreen and saves it in PNG and SVG formats

← Assigns a color to each node group

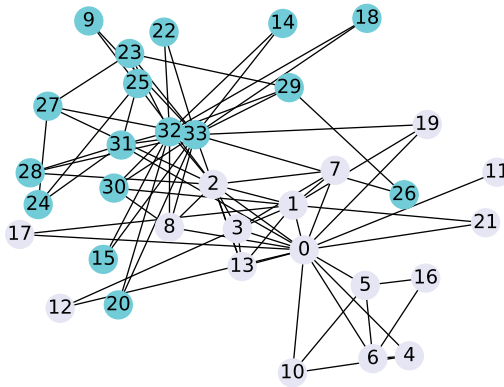


Figure 9.9 The karate club graph produced by the code in listing 9.1, with the shades of the nodes representing the club the member ultimately joined

The resulting network is shown in figure 9.9. The nodes are two different shades, representing the club each member ultimately joined after the split.

As described in figures 9.7 and 9.8, the first step is creating a vector representation of each node that will serve as input for classification algorithms during training and prediction. Chapters 10 and 11 will explore sophisticated embedding techniques in detail; here, let's start with a simple approach: representing each node by its degree (the number of connections it has to other nodes). This feature captures a node's connectivity in the network. The next listing computes the degree of each node in the graph.

Listing 9.2 Representing each node by its degree

```

def compute_degree_embeddings(G):
    embeddings = np.array(list(dict(G.degree()).values()))
    embeddings = [[i] for i in embeddings]
    return embeddings

```

← Computes the embedding for each node

← Converts an element in the list into a single-value embedding

Exercise

Although this technique is foundational, the degree of a node may not offer substantial help in classifying nodes for our goal. As an exercise, try using two other metrics: the degree of Mr. Hi's neighbors and the degree of John A's neighbors. They provide more insight for the algorithm to discern the group to which a node belongs. The solution is shown later in this chapter.

Before the advent of GNNs, Node2Vec [25] was a prominent technique for autonomous representation learning that computed node embeddings based purely on network structure. The following listing shows how to generate these structure-aware node embeddings using the Node2Vec algorithm.

Listing 9.3 Using Node2Vec as an embedding technique

```
def compute_complex_embeddings(G):
    node2vec = Node2Vec(
        G,
        dimensions=64,
        walk_length=30,
        num_walks=200,
        workers=4,
        seed=0)

    model = node2vec.fit(
        window=10,
        min_count=1,
        batch_words=4,
        seed=0)

    embeddings = [model.wv.get_vector(i) for i in G.nodes]
    return embeddings
```

Node2Vec library constructor precomputes probabilities and generates random walks.

Computes embeddings

This code does the following:

- 1 It initializes Node2Vec by specifying that a 64-dimensional vector should represent each node. The algorithm then performs 200 random walks through the graph, each visiting 30 nodes. For efficiency, it uses four parallel processes, and a random seed ensures reproducible results.
- 2 The model is trained using parameters inspired by Word2Vec: it considers 10 nodes before and after each walk sequence, includes all nodes in the training (even those visited once), and processes words in small batches.
- 3 It extracts the learned embeddings for each node in the graph, returning a list of vector representations in which each vector captures the node's structural role in the network. These embeddings can be used as input features for downstream ML tasks like node classification or link prediction.

Listings 9.2 and 9.3 compute the embedding for the full graph, so we don't need to compute it separately for the training set and then for prediction. The following listing shows an example of a training function that uses logistic regression as a classifier.

Listing 9.4 Training function

node_embeddings is a matrix in which each row contains the vector representation (embedding) of a node.

node_labels is a vector containing the class label for each node, aligned with **node_embeddings**.

```
def train(self, train_dataset):
    node_embeddings = train_dataset.embeddings.values.tolist()
    node_labels = train_dataset.label.values.tolist()

    self.scaler = StandardScaler().fit(node_embeddings)
    scaled_embeddings = self.scaler.transform(node_embeddings)

    clf = LogisticRegressionCV(
        random_state=0,
        solver='liblinear',
        multi_class='ovr',
        max_iter=1000)
    self.model = clf.fit(scaled_embeddings, node_labels)
```

Initializes a logistic regression classifier with cross-validation for automatic parameter tuning

Standardizes the embeddings to have zero mean and unit variance

Trains the classifier using the standardized embeddings and their corresponding labels

This training process introduces two important ML concepts: feature standardization and logistic regression. *Feature standardization* is essential when working with multiple node features that operate on different scales. In our example, node degrees may range from single digits to thousands, and other metrics operate on different scales. By using `StandardScaler` to transform all features so they have zero mean and unit variance, we ensure that each feature contributes proportionally to the model's decisions, regardless of its original scale.

Why feature scaling matters

ML algorithms often rely on Euclidean distance calculations between data points. Without proper scaling, features with larger numerical ranges will dominate these calculations, regardless of their actual importance. For example, if node degree ranges from 1 to 1,000 and centrality measures range from 0 to 1, the degree will overwhelm the influence of centrality in distance-based calculations. This can lead to biased predictions and reduced model accuracy, as the model becomes overly sensitive to features simply because of their scale rather than their predictive power.

Logistic regression, despite its name, is a classification algorithm that excels at binary prediction tasks [25]. Here, it estimates the probability of a node belonging to a particular class. The algorithm transforms linear combinations of features into probabilities

between 0 and 1 using the logistic function, making it ideal for our node classification task. For instance, in the karate club example, it predicts the probability of each member joining either the instructor's or administrator's club.

This implementation demonstrates a key advantage of our approach: by converting graph data into feature vectors through embeddings and proper scaling, we can use well-established ML algorithms. As shown in the following listing, the subsequent evaluation phase tests this model's accuracy by comparing predicted labels with actual outcomes for a held-out set of test nodes.

Listing 9.5 Making a prediction and comparing it with the actual value

```

test_embeddings is a matrix in which  
each row contains the vector  
representation of a test node.
def evaluate(self, test_dataset):
    test_embeddings = test_dataset.embeddings.values.tolist()
    true_labels = test_dataset.label.values.tolist()

    true_labels contains the class  
labels for test nodes, aligned  
with test_embeddings.

    Standardizes  
test embeddings  
using the same  
scaler fitted on  
training data
    scaled_test_embeddings = self.scaler.transform(test_embeddings)

    predicted_labels = self.model.predict(scaled_test_embeddings)

    print("True labels:\t\t", true_labels)
    print("Predicted labels:\t", list(predicted_labels))

    Predicts class  
labels for each  
test node using  
the trained model

    # Calculate performance metrics
    metrics = precision_recall_fscore_support(true_labels,
    ➤ predicted_labels, average='weighted')
    print('Precision:', metrics[0], 'Recall:', metrics[1], 'f-score:',
    ➤ metrics[2])
    conf_matrix = confusion_matrix(true_labels, predicted_labels)
    print("Confusion Matrix:\t", conf_matrix)

    Calculates precision, recall, and F1-score by  
comparing predicted labels against true labels

    Generates a confusion matrix to visualize  
prediction accuracy across different classes

```

This function is useful for evaluating the quality of the predictive model. Our example has a very small network and just two labels, but the code also works for bigger graphs and multiple labels.

At this point, we have all the ingredients to perform the training and the evaluation of our node classification task. Listing 9.6 uses all the components (functions) from previous listings and prints the results. Switching from one embedding to the other just requires commenting and uncommenting the desired embedding function.

Listing 9.6 The full node classification process

```

G = nx.karate_club_graph()
draw_and_save_graph_picture(G)

```

```

labels = np.asarray([G.nodes[i]['club'] != 'Mr. Hi' for i in G.nodes])
               .astype(np.int64)

#embeddings = compute_degree_embeddings(G)
embeddings = compute_complex_embeddings(G)

df = pd.DataFrame({
    'nodeId': G.nodes,
    'embeddings': embeddings,
    'label': labels
})

train, test = train_test_split(df, test_size=0.4, random_state=0)

classifier = EvaluateEmbedding()
classifier.train(train)
classifier.evaluate(test)

```

Retrieves the label for each node and assigns a value of 0 or 1 depending on the group it belongs to

Select the embedding to test by uncommenting the option you wish to try.

Creates a DataFrame that includes the nodeId, the full embedding, and the node labels

Invokes the functions to create and evaluate

Divides the DataFrame into two frames by randomly splitting the dataset

After running the code with the two different embedding techniques, we can see the different results in the following listing.

Listing 9.7 Results for the two embeddings

RESULTS WITH MORE SIMPLE EMBEDDINGS USING DEGREE

```

Gold:      [0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0]
Predicted: [1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0]

```

```

Precision: 0.44642857142857145
Recall:    0.42857142857142855
f-score:   0.42857142857142855

```

```

Confusion Matrix:  [[3 3]
                   [5 3]]

```

Confusion matrix for first run: three true negatives (top-left), three false positives (top-right), five false negatives (bottom-left), and three true positives (bottom-right)

RESULTS WITH MORE COMPLEX EMBEDDINGS USING NODE2VEC

```

Gold:      [0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0]
Predicted: [0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0]

```

```

Precision: 0.6530612244897959
Recall:    0.6428571428571429
f-score:   0.6446886446886447

```

```

Confusion Matrix:  [[4 2]
                   [3 5]]

```

Confusion matrix for second run: four true negatives, two false positives, three false negatives, and five true positives

The results from both runs demonstrate relatively poor and inconsistent performance. Multiple executions of these scenarios show significant volatility in the outcomes, with *f*scores fluctuating widely between 30% and 70%. Given that logistic regression is a well-established and proven algorithm, this instability suggests that the limitation lies not in the classification method but in our feature engineering approach. The high variance in results indicates that our simple node representation,

based solely on degree, fails to capture the complex network patterns necessary for reliable node classification. The same is true with Node2Vec.

Let's improve our feature engineering by using homophily. Instead of using only node degree, which simply counts total connections, we can create a richer node representation by considering the social context of these connections. For each node, we'll create a three-element vector containing the total degree (total connections), the "Mr. Hi degree" (connections to the instructor's group), and the "officer degree" (connections to the administrator's group). This approach, shown in the next listing, capitalizes on homophily by assuming that a node's group membership is likely influenced by the group affiliations of its neighbors.

Listing 9.8 Computing featurization based on three types of degrees

```
def compute_specific_degree_embeddings(G):
    clubs = nx.get_node_attributes(G, "club")
    mr_hi_degree =
        [[clubs[c] for c in G.neighbors(i)].count('Mr. Hi') for i in
         G.nodes()]
    officer_degree =
        [[clubs[c] for c in G.neighbors(i)].count('Officer')
         for i in G.nodes()]
    degree = list(dict(G.degree()).values())
    embeddings =
        [[degree[i], mr_hi_degree[i], officer_degree[i]] for i in G.nodes]
    return embeddings
```

Computes the degree in the traditional way, considering all nodes →

Calculates mr_hi_degree, taking into account only neighbors that belong to the Mr. Hi group ←

Calculates officer_degree, taking into account only neighbors that belong to the Officer group ←

Combines the three values into a single vector for each node ←

Now that we have a new function, we just need to change listing 9.6 to point to the new embedding function. The results are presented in the next listing.

Listing 9.9 Results with the vectors made using three degrees

```
Gold:      [0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0]
Predicted: [0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1]

Precision: 0.9365079365079365
Recall:    0.9285714285714286
f-score:   0.9274255156608098

Confusion Matrix:  [[5 1]
                   [0 8]]
```

The results are much better. You can run this multiple times, and the results will be very stable and always close to 100%.

This experiment, although simple, demonstrates several principles for building effective IASs using ML on graphs:

- *Feature engineering is critical.* The way we represent nodes and relationships fundamentally impacts the success of graph-based ML tasks, with domain-informed features often outperforming simple metrics like degree centrality.

- *Autonomous embeddings require careful tuning.* Methods like Node2Vec, although powerful, don't guarantee optimal results. Parameters must be thoughtfully configured to avoid generating overly homogeneous node representations. As an exercise, run the experiment using lower values of `walk_length` and `num_walks`.
- *Domain understanding matters.* Knowledge of underlying graph dynamics and network properties like homophily can guide better feature engineering strategies that are often more effective than generic approaches.

These insights will serve as foundations as we explore more sophisticated approaches to graph representation learning in subsequent chapters.

9.3.2 Graph classification

The approach to feature engineering, discussed before for node classification and link prediction, is similar for graph classification. But instead of extracting features for nodes, most algorithms compute or extract a set of features for the entire graph—and, of course, we have multiple graphs as input. Figure 9.10 shows the high-level process.

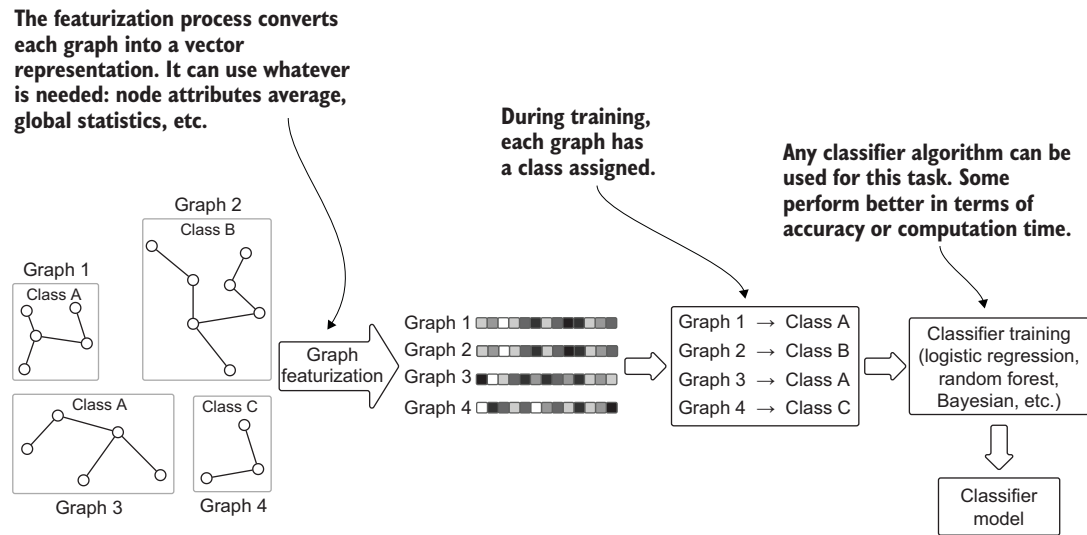


Figure 9.10 High-level training process for graph classification. The output is a classifier model that has been trained on the known classes of different graphs.

Once the model has been trained, it is used as an input for the prediction phase together with the unclassified graphs. Figure 9.11 shows the key steps and actors of the prediction.

The featurization process must be the same as during training and must be consistent.

The graph classification must predict the class of the graph. The input is the graph's features.

The output for the node classifier is the probability that a node belongs to a class. A threshold allows the system to decide on the right class.

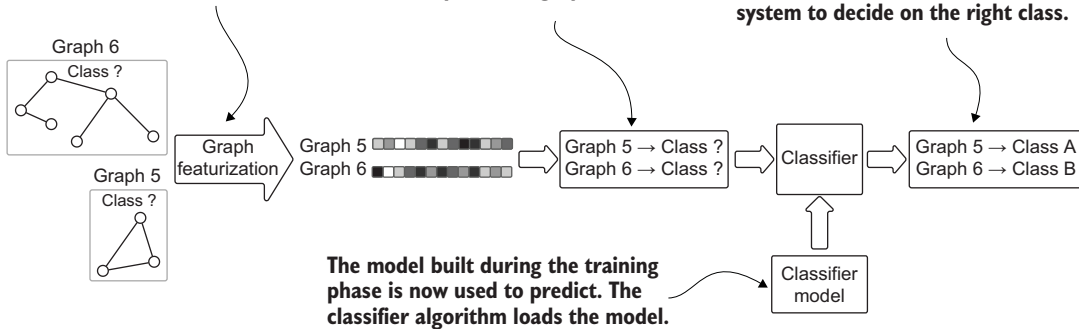


Figure 9.11 High-level prediction process for graph classification. The outputs are the classes assigned to the unlabeled graphs.

Again, it is clear that feature extraction for nodes, relationships, and graphs is a vital aspect of the process because it forms the inputs for training and prediction. The final quality of the output, including accuracy and overall performance, largely depends on the quality of these features and the precise tuning of downstream algorithm parameters. Chapter 10 and much of the rest of the part 4 will focus on this task and using the results of this process effectively.

9.3.3 Graph clustering

Not all ML tasks on graphs require the steps shown in the previous sections. In the case of graph clustering, for example, the input is the entire graph (or a subgraph), and the algorithms use nodes and relationships to extract communities (see figure 9.12).

Community detection identifies groups of nodes with denser internal connections compared to their connections with the rest of the network. It maximizes modularity by optimizing the difference between actual and expected edge density within communities.

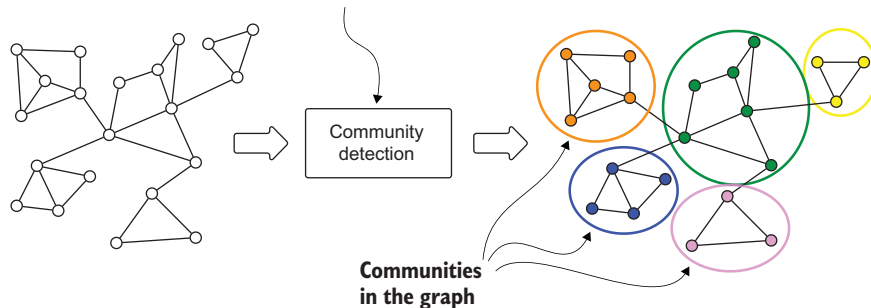


Figure 9.12 High-level graph clustering process. This is an example of a pure graph algorithm approach that doesn't require an intermediate transformation into a vector representation or other format.

For this task, there is no feature extraction because graph algorithms use node relationships, their directions, and their weights to split the graph. Certain algorithms, such as weakly connected component (WCC), use a fixed mechanism and consider only independent subgraphs. Other methods, such as the Louvain and label propagation algorithms, optimize certain outcomes, such as modularity. We discussed Louvain in chapter 4; let's quickly introduce label propagation.

The *label propagation algorithm* (LPA) [20] is a fast method for detecting communities in a network. It propagates labels throughout the network, and at the end, nodes with the same labels are considered to belong to the same community. LPA doesn't guarantee consistent output, due to its randomness; therefore, running it multiple times on the same network may produce slightly different communities.

To show how the process works, we'll use the same karate club graph and run Louvain on it using the following code.

Listing 9.10 Running Louvain on the karate club network

```
import math
import time
import networkx as nx
import matplotlib.pyplot as plt

def set_club_colors(G):
    for node in G.nodes(data=True):
        color = '#00fff9'
        if node[1]['club'] == 'Mr. Hi':
            color = '#e6e6fa'
        node[1]['color'] = color

def draw_and_save_graph_picture(G, i=0):
    set_club_colors(G)
    layout_position = nx.spring_layout(G, k=8 / math.sqrt(G.order()))
    colors = [n[1]['color'] for n in G.nodes(data=True)]
    nx.draw_networkx(G, pos=layout_position, node_color=colors)
    plt.axis('off')
    plt.savefig("Karate_Graph_" + str(i) + ".svg", format="SVG", dpi=1000)
    plt.savefig("Karate_Graph_" + str(i) + ".png", format="PNG", dpi=1000)
    plt.show()

if __name__ == '__main__':
    start = time.time()
    G = nx.karate_club_graph()
    draw_and_save_graph_picture(G)
    communities = nx.community.louvain_communities(G, seed=123)
    i = 1
    for community in communities:
        subGraph = G.subgraph(community)
        draw_and_save_graph_picture(subGraph, i)
        i += 1

    end = time.time() - start
    print("Time to complete:", end)
```

Computes communities using Louvain

←

Draws multiple graphs, one for each community

←

This code identifies four communities, as shown in figure 9.13. We immediately see that the algorithm did a great job identifying groups of nodes which are well connected with other nodes in the same community and loosely connected with nodes outside. Each community is pretty much homogeneous, with nodes belonging to the same group (as indicated by the shades of the nodes). Only in one case do we have a spurious node (number 8), which is in a community with nodes that are all from the other group. Maybe this person is in the wrong group!

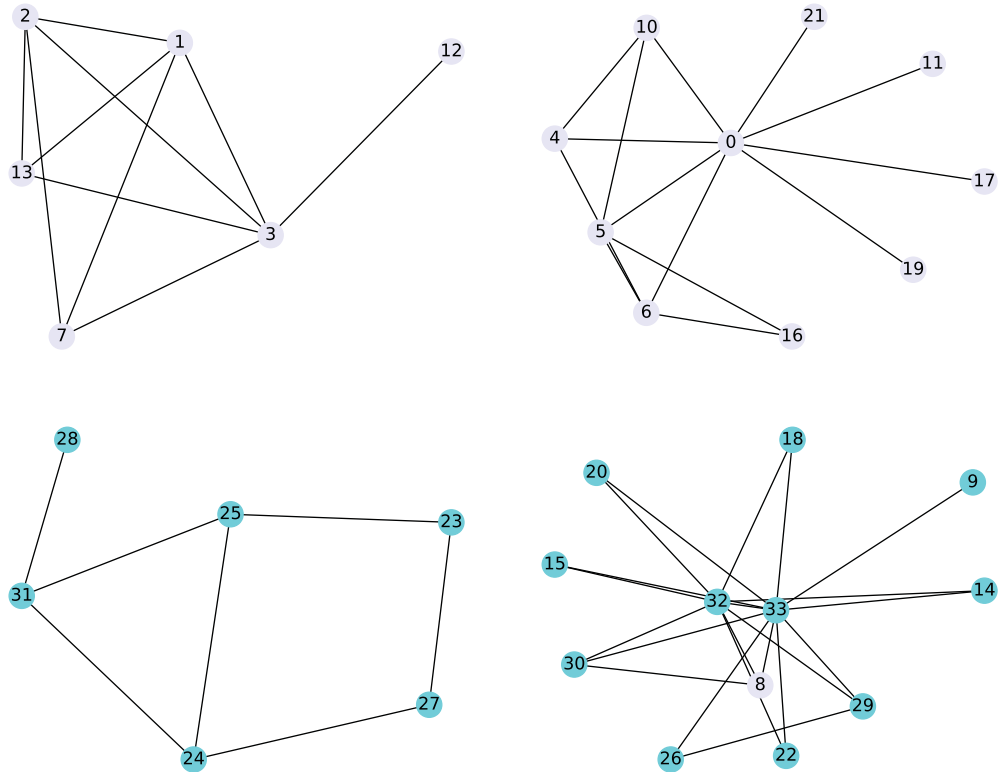


Figure 9.13 The result of the community detection algorithm applied to the karate club network. The algorithm identified four communities. All but one contain only members of the same groups. There is one spurious node, but it is well connected to other people belonging to a different group.

Two aspects distinguish this task from those we’ve covered previously. First, there’s no requirement for representation transformation. The entire graph serves as the input, and the algorithm directly interacts with the nodes and relationships. And second, there’s no training phase. This task follows an unsupervised approach. It takes the graph and generates the output without using any examples of labels, etc.

Summary

- Machine learning on graphs naturally handles interconnected data, provides universal data representation, and enables the modeling of complex problems through a limited set of computational tasks.
- Unlike traditional ML, which assumes independent and identically distributed data points, graph-based approaches use connections and dependencies between nodes, better reflecting real-world relationships and patterns.
- ML tasks on graphs fall into two main categories: node-focused (like node classification and link prediction), where tasks are performed on a single graph, and graph-focused (like graph classification), where each graph is a separate data point.
- Node classification and link prediction are typically semisupervised tasks, using both labeled and unlabeled data, along with the graph's structural information. Community detection is usually unsupervised and operates directly on the graph structure.
- Feature engineering plays a critical role in these tasks.
- Node embeddings can be generated through various approaches, from manual feature engineering to automated techniques like Node2Vec. However, autonomous embeddings require careful tuning to avoid generating overly homogeneous representations.
- The success of graph-based ML often depends on finding the right balance between automated feature learning, domain knowledge incorporation, and appropriate algorithm selection for the task at hand.